

Selected ADA Programs

Floyd's Algorithm

Floyd's Algorithm:

```
#include <stdio.h>

int min(int a, int b) {
    return (a < b) ? a : b;
}

void floyd(int dist[10][10], int n) {
    for (int k = 0; k < n; k++)
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
                dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j]);
}

int main() {
    int n, dist[10][10];
    printf( "Enter number of vertices: ");
    scanf("%d", &n);

    printf( "Enter adjacency matrix (999 for no direct edge):\n");
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            scanf("%d", &dist[i][j]);

    floyd(dist, n);

    printf("All-Pairs Shortest Paths:\n");
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++)
            printf("%d ", dist[i][j]);
        printf("\n");
    }

    return 0;
}
```

Warshall's Algorithm

Warshall's Algorithm:

```
#include <stdio.h>

void warshall(int p[10][10], int n) {
    for (int k = 0; k < n; k++)
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
                p[i][j] = p[i][j] || (p[i][k] && p[k][j]);
}
```

```

}

int main() {
int n, a[10][10];
printf("Enter number of vertices: ");
scanf("%d", &n);

printf("Enter adjacency matrix (1 if edge exists, else 0):\n");
for (int i = 0; i < n; i++)
for (int j = 0; j < n; j++)
scanf("%d", &a[i][j]);

warshall(a, n);

printf("Transitive Closure (Path Matrix):\n");
for (int i = 0; i < n; i++) {
for (int j = 0; j < n; j++)
printf("%d ", a[i][j]);
printf("\n");
}

return 0;
}

```

Dijkstra's Algorithm

Dijkstra's Algorithm:

```

#include <stdio.h>

void dijkstra(int cost[10][10], int n, int src, int dist[]) {
int visited[10] = {0}, min, u;

for (int i = 0; i < n; i++)
dist[i] = cost[src][i];

visited[src] = 1;

for (int i = 1; i < n; i++) {
min = 999;
for (int j = 0; j < n; j++)
if (!visited[j] && dist[j] < min) {
min = dist[j];
u = j;
}
visited[u] = 1;

for (int v = 0; v < n; v++)
if (!visited[v] && dist[u] + cost[u][v] < dist[v])
dist[v] = dist[u] + cost[u][v];
}

int main() {

```

```

int n, cost[10][10], dist[10], src;
printf("Enter number of vertices: ");
scanf("%d", &n);
printf("Enter cost adjacency matrix (use 999 for no edge):\n");
for (int i = 0; i < n; i++)
for (int j = 0; j < n; j++)
scanf("%d", &cost[i][j]);

printf("Enter source vertex (0 to %d): ", n - 1);
scanf("%d", &src);

dijkstra(cost, n, src, dist);
printf("Shortest distances from source %d:\n", src);
for (int i = 0; i < n; i++)

printf("To %d = %d\n", i, dist[i]);

return 0;

```

N-Queens

N-Queens:

```

- OX
#include <stdio.h>
int board[10], count = 0;
int isSafe(int row, int col) {
for (int i = 0; i < row; i++)
if (board[i] == col || abs(board[i] - col) == abs(i - row))
return 0;
return 1;
}

void display(int n) {
printf("\nSolution %d:\n", ++count);
for (int i = 0; i < n; i++) {
for (int j = 0; j < n; j++)
printf("%c ", board[i] == j ? 'Q' : 'x');
printf("\n");
}

void solve(int row, int n) {
for (int col = 0; col < n; col++) {
if (isSafe(row, col)) {

board[row] = col;

if (row == n - 1)
display(n);

else
solve(row + 1, n);
}
}
}

```

```

}

int main() {
int n;
printf("Enter number of queens: ");
scanf("%d", &n);
solve(0, n);
printf("Total solutions = %d\n", count);
return 0;

```

Subset Sum Problem

Subset Sum Problem:

```

#include <stdio.h>
int w[10], x[10], n, target;

void subset(int s, int k, int r) {
x[k] = 1;
if (s + w[k] == target) {
printf("Subset: ");
for (int i = 0; i <= k; i++)
if (x[i])
printf("%d ", w[i]);
printf("\n");
} else if (s + w[k] + w[k + 1] <= target)
subset(s + w[k], k + 1, r - w[k]);

if ((s + r - w[k] >= target) && (s + w[k + 1] <= target)) {
x[k] = 0;
subset(s, k + 1, r - w[k]);
}

int main() {
int sum = 0;
printf("Enter number of elements: ");
scanf("%d", &n);

printf("Enter elements in ascending order:\n");
for (int i = 0; i < n; i++) {

scanf("%d", &w[i]);

sum += w[i];

}

printf("Enter target sum: ");
scanf("%d", &target);

if (sum < target || w[0] > target) {
printf("No solution possible.\n");

```

```

return 0;

}

subset(0, 0, sum);
return 0;

```

0/1 Knapsack DP

0/1 Knapsack DP:

```

#include <stdio.h>

int w[10], p[10], dp[10][10], x[10];
int n, capacity;

int max(int a, int b) {
return (a>b)? a:b;
}

void knapsack() {
for (int i = 0; i <n; i++) {
for (int j = 0; j <= capacity; j++) f{

if (i == 0 || j == 0)
dp[i][j] = 0;

else if (w[i] > j)
dp[i][j] = dp[i - 1][j];

else
dp[i][j] = max(dp[i - 1][j], dp[i - 1][j - w[i]] + p[i]);

}

void findItems() {
int i = n, j = capacity;
while (i > 0 & j > 0) {
if (dp[i][j] != dp[i - 1][j]) {
x[i] = 1;
j -= w[i];
}
i--;
}

int main() {
printf("Enter number of items: ");
scanf("%d", &n);
printf("Enter knapsack capacity: ");
scanf("%d", &capacity);

printf("Enter weights:\n");
for (int i = 1; i <= n; i++)

```

```

scanf("%d", &w[i]);

printf("Enter profits:\n");
for (int i = 1; i <= n; i++)
scanf("%d", &p[i]);

knapsack( );
findItems();

printf("Items included:\n");
printf("Item\tWeight\tProfit\n" );
for (int i = 1; i <= nn; i++)
if (xf[i])
printf("%sd\t%d\t%d\n", i, wli, p[i]);

printf("Max Profit = %d\n", dp[n][capacity]);
return 0;

```

Discrete & Continuous Knapsack (Greedy Method)

Discrete & Continuous Knapsack (Greedy Method):

```

_ x

#include <stdio.h>

void greedyKnapsack(int n, float w[], float p[], float capacity) {
float x[20] = {0}, ratio[20], totalProfit = 0;
int i, j;

// Compute profit/weight ratio
for (i = 0; i <n; i++)
ratio[i] = p[i] / wli];

// Sort items by ratio (descending)
for (i =0; i<n-1; i++) {
for (j =i+1; j <n; j++) f{
if (ratio[i] < ratio[j]) f{
float temp;

// Swap ratio
temp = ratio[i]; ratio[i] = ratio[j]; ratio[j] = temp;

// Swap weights
temp = w[i]; w[i]

w[j]; wlj] = temp;

// Swap profits
temp = pli]; pli]

plj]; plj] = temp;

}

```

```

// Fill the knapsack
for (i = 0; i <n; i++) {
    if (w[i] <= capacity) {
        x[i] = 1.0; // Take full item
        totalProfit += p[i];
        capacity -= w[i];
    } else {
        x[i] = capacity / w[i]; // Take fraction
        totalProfit += x[i] * p[i];
        break; // Knapsack is full
    }

// Result
printf("\nItem\tWeight\tProfit\tFraction taken\n");
for (i = 0; i <n; i++)
    printf ("%s\t%.1f\t%.1f\t%.2f\n", i +1, w[i], p[i], x[i]);

printf("\nTotal Profit = %.2f\n", totalProfit);
}

int main() {
    int n;
    float w[20], p[20], capacity;

    printf("Enter number of items: ");
    scanf("%d", &n);

    printf("Enter knapsack capacity: ");
    scanf("%f", &capacity);

    printf("Enter weights and profits:\n");
    for (int i = 0; i <n; i++)
        scanf("%f%f", &w[i], &p[i]);

    greedyKnapsack(n, w, p, capacity);
    return 0;
}

```